The Relay — Evaluation Report

1 What I built and why.

A Python RAG pipeline over only the required versions: *Attention Is All You Need* (1706.03762v7), *Moshi* (2410.00037v1), and *Dynamic Chunking* (2507.07955v2). It extracts page-aware chunks, retrieves with TF-IDF via a tool call, sends evidence to Groq (`openai/gpt-oss-20b`), and maps evidence tags to paper/page citations. Weak retrieval or missing/invalid tags trigger refusal. A FastAPI stand-in simulates latency, jitter, streaming, failures, RPM/daily/in-flight admission limits and overload slowdown.

2 The real service and my stand-in.

The account dashboard showed the limits below; they were not stress-tested. Five sequential, non-streaming real calls succeeded: 471 input + 429 output tokens total; latency 298.05–856.74 ms (median 318.36 ms). The stand-in is an approximation, not a Groq replica: it rejects configured limits with HTTP 429, simulates 503/504, and streams after delay. It has no queue (peak queue 0); no timeout occurred.

Table 1 — Real-service limits	Dashboard	How found / confidence
RPM / TPM	30 / 8,000	Account dashboard; medium
RPD / TPD	1,000 / 200,000	Account dashboard; medium
In-flight	Not measured	No real concurrency test
Limit response / recovery	Not measured	No real quota exhaustion

3 Results.

Percentiles use nearest-rank. Offered rate is requests / first-start-to-last-completion window; short burst windows are not sustained 60-second rates. TTFT is unavailable for non-streaming Groq calls.

Table 2a — Load/latency (TTFT and total p50/p95/p99, ms)

Scenario (source)	N	Duration s	Offered/min	TTFT	Total
Steady (stand-in)	10	10.471	57.30	677/737/737	1390/1455/1455
Burst (stand-in)	20	2.203	544.81	47/1337/1448	52/2027/2151
Sustained overload	40	6.236	384.85	12/1404/1419	15/2107/2136
Half-RPM	35	0.953	2204.41	145/160/161	866/880/881
Daily budget	11	2.607	253.18	65/142/142	765/820/820
Retry (stand-in)	20	3.658	328.09	N/M	96/659/666
Sequential (real)	5	10.309	29.10	N/M	318/857/857

Table 2b — Outcomes

Scenario	Success	429	Timeout	Other err.	Retries/req	Queue	Tokens/answer; cost/1k
Steady	100%	0%	0%	0%	0	0	N/A; not measured
Burst	40%	60%	0%	0%	0	0	N/A; not measured
Sustained	50%	50%	0%	0%	0	0	N/A; not measured
Half-RPM	82.86%	14.29%	0%	2.86%	0	0	N/A; not measured
Daily budget	81.82%	9.09%	0%	9.09%	0	0	N/A; not measured
Retry	100% eventual	0% final	0%	0% final	0.15	0	N/A; not measured
Real sequential	100%	0%	0%	0%	0	N/M	180 avg (94.2 in, 85.8 out); paid rate N/M

Two stand-in errors (one half-RPM, one daily-budget) lacked status detail. Three of 20 retry requests first received 503 and recovered after one retry. No timeout occurred; static stand-in replies have no token counts.

4 Where the time goes.

Slowest steady stand-in request `09eea129…b1a6b1`: TTFT 737.20 ms; total 1454.52 ms; 717.32 ms elapsed after first token; retries 0. No queue-wait or model-compute telemetry exists: pre-token time combines simulated delay/transport; post-token time includes streamed word delays. Real TTFT was not measured.

The Relay — Evaluation Report (continued)

5 When a limit is hit.

The stand-in daily-budget run accepted 10 requests; later admission returned HTTP 429 `simulated_daily_limit`. With no queue, excess requests are rejected, not delayed; no weaker answer is served. At 3 pm with the budget exhausted, the caller gets a rate-limit error, not an answer. A friendly UI message/deferred delivery is not implemented. Real Groq limit response/recovery remain unmeasured.

Retrieval: quality versus cost.

45 real pipeline calls tested 15 answerable questions per setting; answers and cited pages were manually checked against the PDFs.

Table 3 — Retrieval quality (answer generation; no reranker)

Chunk/top-k	Rerank	Correctly cited	Wrong	Correctly refused	Avg tokens	TTFT p50/p95	Total p50/p95 ms
150 / 3	No	15/15 (100%)	0%	N/M (answerable)	1,138	N/M	706/1,633
220 / 5	No	13/15 (86.67%)	13.33%	N/M (answerable)	2,023	N/M	800/1,797
350 / 5	No	14/15 (93.33%)	6.67%	N/M (answerable)	2,717	N/M	877/1,086

150/top-3 was fastest/cheapest in this sample; 350/top-5 was selected as the production compromise (one audited error versus two for 220/top-5, with a wider evidence window). Only 15 questions/setting: no broad statistical claim. TTFT was not measured (non-streaming); refusal rate was not measured because all 45 questions were answerable.

Separate top-1 retrieval/reranking test (15 questions per size): baseline vs reranked was 14/15 vs 13/15 (150 words), 14/15 vs 14/15 (220), and 14/15 vs 15/15 (350). Reranking hurt at 150, was unchanged at 220, and helped at 350. These are retrieval top-1 results, not answer-quality results; the 45 generation evaluations used no reranker. Production retains no reranker because end-to-end answer quality with reranking was not tested.

How this library will mislead a naive retriever.

1) **Keyword overlap:** Transformer encoder/decoder query A05 ranked a Dynamic Chunking passage first although the relevant Transformer passage was in the top three. 2) **Right paper, wrong passage:** Moshi query M01 reranked a toxicity/regurgitation passage above architecture evidence. 3) **Paper hit ≠ answer passage:** Moshi/Mimi query M03 first retrieved a passage naming Mimi as a codec but not explaining its role; a later passage was needed.

6 Retries.

The client retries 429/503 up to twice (0.5 s then 1 s backoff), not timeouts/other errors. Three of 20 logical requests recovered after one retry (0.15 retries/request). A recovered 503 incurs 0.5 s backoff; this demonstrates added delay, not a measured no-retry counterfactual.

7 What the person sees.

Supported questions show answers with paper/version/page citations; weak evidence triggers refusal. H100 price in India and India's 2026 population were refused; population was rejected before a model call (zero tokens). At stand-in daily exhaustion, the prototype exposes HTTP 429, not a friendly budget message or queue.

8 What I'm unsure about, what I left out, what I learned.

Five real calls are too few to represent normal latency/concurrency. The stand-in has no queue and cannot reproduce Groq scheduling/quota windows. I omitted real quota exhaustion and paid-cost projection to avoid quota circumvention and unsupported estimates. I learned that valid citation tags do not prove claim entailment; page review caught errors tag validation misses.

Your feedback on Vaani.

The public site (vaaani.dev) presents a clear build/humanise/connect/run/improve/scale voice-agent workflow. I would publish benchmark conditions (hardware, network, model, sample size, percentile definition) beside latency claims and make developer onboarding/docs easier to find. This is public-site feedback, not paid-product use.

How I worked with the assistant.

I used the assistant for implementation/debugging and experiment interpretation. Three representative prompts (summarised, not verbatim): build page-aware RAG over the exact papers; implement a local stand-in for latency/streaming/limits/failures; compare chunk sizes, reranking, answer quality and load. I caught a premature 45/45 “all correct” assessment: page checks found errors in 220-word M04/D02 and 350-word D02, correcting grades to 15/15, 13/15 and 14/15. A model-only answer called Mimi a fictional character; the Moshi paper identifies it as a neural audio codec. I overruled enabling the heuristic reranker because paper-level scores did not prove passage relevance. My work: Windows/Groq setup, PDF download/indexing, real and simulated runs, evidence/page/CSV inspection and final configuration selection.